A CASE STUDY REPORT

ON

Library Book Search Using Random File Access (fseek,ftell)

Submitted by

25RH1A05K1 D.Rekha
25RH1A05K2 D.Akshitha
25RH1A05K3 D.Deepika
25RH1A05K4 D.Amulya

I B. Tech

MALLA REDDY ENGINEERING COLLEGE FOR WOMEN

Autonomous Institution, UGC, Govt. of India
Programmes Accredited by NBA
Accredited by NAAC with A+ Grade
Affiliated to JNTUH, Approved by AICTE, ISO 9001:2015 Certified Institute
Maisammaguda (V), Dhullapally (Post), (Via) Kompally, Medchal Malkajgiri Dist. T.S-500100

DECEMBER-2025

INDEX

Title	Page no
CASE STUDY	i
1. Analysis	1-3
2. Algorithm	4
3. Model Diagram	5
4. Implementation	6-8
5. Result	9

CASE STUDY

Managing a large digital library requires fast and efficient access to book information. In many beginner-level systems, book records are stored in files and searched sequentially. This means the program must read each record one by one until it finds the required book. As the number of books grows, this method becomes slow and inefficient. To overcome this limitation, random file access techniques using fseek() and ftell() in C programming are used.

Random file access allows the program to directly jump to any position inside the file without reading unnecessary data. This is useful when all book records are stored in fixed-size structures. The position of each record can be calculated mathematically or stored in an index file. The ftell() function is used to determine the current file pointer position, which helps in identifying the exact byte address of a book record. Meanwhile, fseek() helps the program move the file pointer directly to the required record's location.

In this case study, a library stores thousands of book records such as book ID, title, author, and price. When a user wants to search for a book by its ID, the system uses an index table that maps each book ID to its file position (byte address). Instead of scanning the entire file, the program uses fseek() to jump directly to that record and retrieve its details. This reduces search time from several seconds to almost instant access.

The implementation using fseek() and ftell() proved significantly faster than sequential searching. It also demonstrated how random access can improve the performance of file-based databases. The approach is efficient, scalable, and suitable for large library applications, although it requires fixed-size records and careful index management.

Overall, this case study shows that using random file access with fseek() and ftell() is an effective technique for building an efficient library book search system**.**

ANALYSIS

1. Introduction

Libraries contain a large collection of books, journals, and reference materials that must be organized and accessed efficiently. With the increasing digitization of library management systems, the need for faster and more reliable methods of accessing book information has become essential. Many traditional systems rely on sequential file access, where the program must read records one by one until the target record is found. While this method works for smaller datasets, it becomes increasingly slow and inefficient as the file size grows.
To solve this problem, the concept of random file access is introduced using the C programming language's functions fseek() and ftell(). These functions allow direct navigation to a specific location within a file, making it possible to retrieve information much more quickly. By combining random access with properly structured data storage, we can create a system that improves search speed, reduces resource consumption, and provides a more scalable solution for library environments.
Thus, the system "Library Book Search Using Random File Access" aims to modernize the searching operation in library management by using efficient file handling techniques.

2. Problem Definition

Libraries typically maintain a large number of books, and efficient searching is a fundamental requirement. In many older or basic systems, books are stored in text files or binary files without any indexing mechanism. When a user searches for a specific book—by ID, title, or author—the program reads every record sequentially from the start of the file. This creates several issues:

- The search process becomes very slow as the file size increases.
- It consumes unnecessary CPU time and I/O operations.
- It results in delays for the user and reduces system efficiency.

The core problem is that sequential access cannot scale well with large or growing datasets. Thus, the problem is defined as:
“To design and implement an efficient library book search system that avoids sequential scanning by using random file access techniques, enabling direct and quick retrieval of book records.”

3. Purpose of the System

The primary purpose of this system is to provide a faster and more efficient method to locate specific books from a large file containing numerous book records. Additional objectives include:

- To demonstrate the practical use of fseek() and ftell() in real-world applications.
- To reduce the time complexity of searching from *O(n)* (linear search) to *O(1)* (direct access) in cases where indexing or fixed record size is used.
- To create a structured and optimized file-based database for books.
- To decrease the overall workload of library staff and improve user satisfaction.
- To encourage the development of more advanced file-handling systems using data structures and indexing.

This system serves both as a functional library tool and an educational project for understanding random access file handling in C.

4. Existing System (Extended)

Most traditional library search systems—especially those built using simple C programs—use sequential file access, where each record is read in order until the desired book is found. These systems:

- Store book records in plain text or binary files.
- Do not maintain any indexing or positional information.
- Require scanning from the beginning of the file each time a search is made.
- Are easy to implement but lack scalability.

In addition, some manual library systems still use physical card catalogs or handwritten logs where the staff must search manually, making the process slow and prone to human error.
Existing digital systems often have the following characteristics:

- File handling is done line-by-line.
- No mechanism to jump to a particular record.
- Searching consumes unnecessary time as the dataset grows.
- Performance is acceptable only for very small collections.

5. Drawbacks of the Existing System

1. LowEfficiency:
   Sequential search becomes slower as the number of records increases, leading to performance bottlenecks.

2. HighI/OOverhead:
   The system continuously reads from the disk without knowing the record's exact position.

3. PoorScalability:
   If the library expands its collection, the search time grows proportionally.

4. Time-consumingOperations:
   Especially when searching for a book located near the end of the file.

5. HigherCPUUtilization:
   The system wastes CPU cycles in unnecessary data reading and comparisons.

6. LackofRandofAccess:
   The system cannot jump to a specific record directly, forcing repeated scanning.

7. DifficulttoMaintainLargeDatabases:

   With increased data size, manual or sequential systems become harder to manage.

6. Proposed System

The proposed system introduces random file access to improve search efficiency. By storing book records in fixed-size structures, the system can calculate the position of any given record and move the file pointer to that location instantly using fseek().
Key Characteristics

- Each record is stored with a fixed length (e.g., struct in C).
- The record number is used to compute the offset: position = (record_number * sizeof(record));
- The function fseek() moves the pointer to this exact location.
- The function ftell() can track the current pointer position and help manage indexing.
- Access time remains constant no matter how large the file grows.

Advantages of the Proposed System

- Significantly faster search time.
- Reduced file scanning and lower I/O operations.
- Simplified record navigation.
- Easily maintainable as the library collection grows.
- More suitable for real-time applications and large datasets.

Functional Features

- Add new book records.
- Search for books using direct access.
- Display or modify book information.
- Jump to any record using random access.
- Maintain a structured binary file system.

This system enables libraries to modernize their search operations with minimal resource usage and maximum efficiency.

8. Scope of the System

The scope of this project is broad and covers both academic and practical library management needs.
In the Present System

- Helps students understand advanced file handling concepts in C.
- Provides a working model of random access file operations.
- Enhances search functionalities in small to medium library systems.
- Can be integrated with basic library software or desktop applications

Future Expansion

The proposed system can be extended to support:

1. GraphicalUserInterface(GUI)
   To make the system more user-friendly
   .
2. DatabaseIntegration
   Migrating from file-based storage to SQL/NoSQL databases.
   1. Network-Based Library
   2. Management
      Allowing multi-user access via LAN or the Internet

   3. Barcode/QRCodeScanning
      For instant book identification and inventory management.

   4. Sorting,Filtering&AdvancedSearchAlgorithms
      For smarter book retrieval based on multiple criteria.

   5. Cloud-basedLibrarySearchSystems
      For large institutions or public libraries.

The system provides an excellent foundation for buildin

ALGORITHM

1.**Start**

2.Open File: Open the book data file in binary read mode ("rb" or "rb+").

3.Check File Status: Verify the file opened successfully. If not, display an error and exit.

4. Get User Input: Prompt the user to enter the Book ID (record number) to search for.

5. Calculate Offset: Calculate the byte offset of the desired record using the formula: offset = Book_ID - 1) * sizeof(struct Book).

6. Position Pointer: Use fseek(file_pointer, offset, SEEK_SET) to move the file pointer to the calculated position.

7. Check Position: Optionally, use ftell() to confirm the pointer's current position.

8. Read Record: Use fread() to read the book record structure at the current pointer position into a memory buffer.

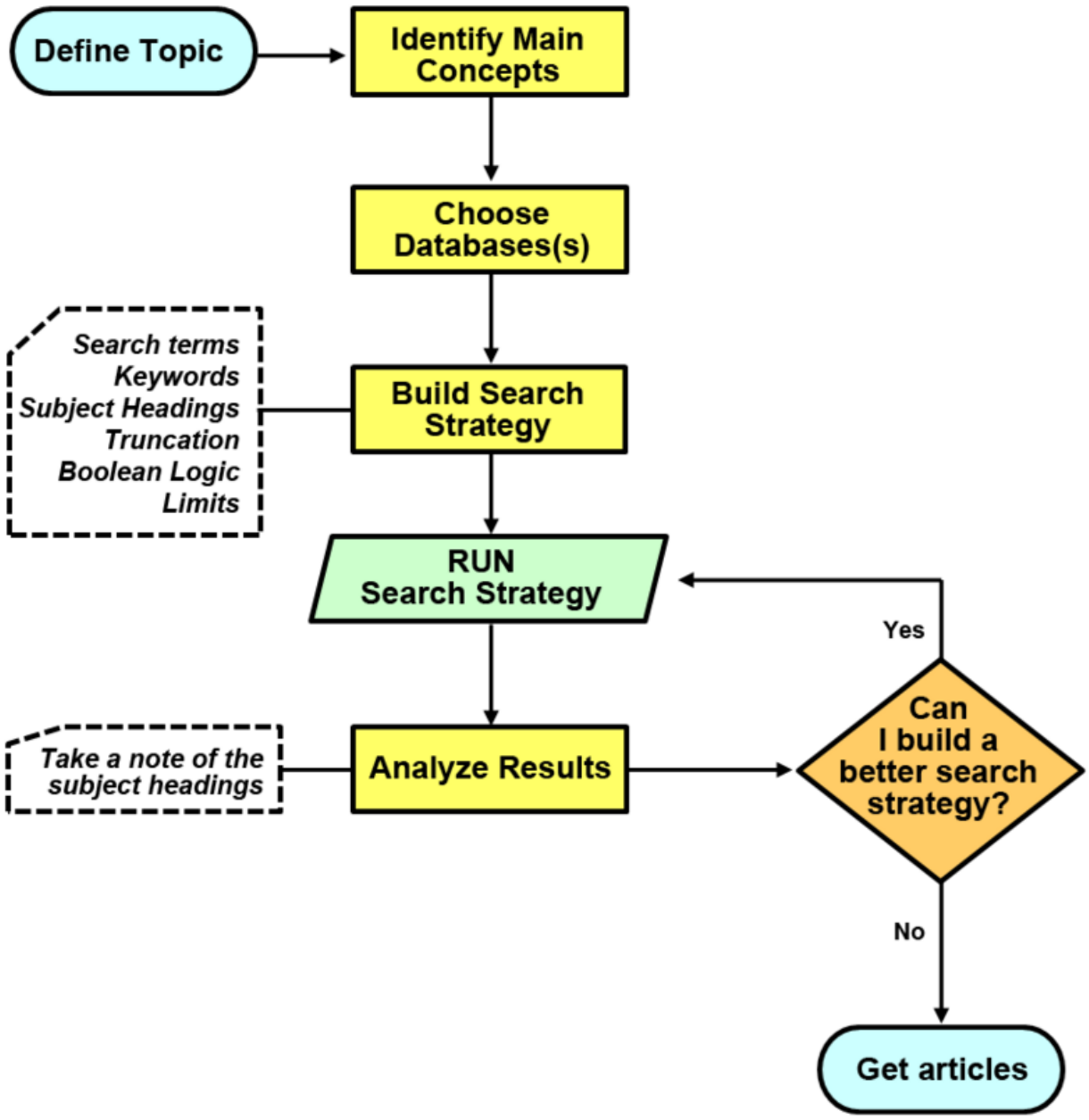
9. Check Read Status/EOF: Verify if the read operation was successful and the end of the file has not been reached. If unsuccessful or beyond the file end, indicate the record was not found.

10 Display Information: If the record is found, display the book's details (title, author, ISBN, etc.).

11.Close File: Close the file using fclose().

12.End

MODEL DIAGRAM:



IMPLEMENTATION

```
#include <iostream>
#include <fstream>
#include <cstring>
#include <iomanip>

using namespace std;

// Define a fixed-size structure for a book record
struct Book {
    int id;
    char title[30];
    char author[30];
    int quantity;
};

// Function to write initial records to the file
void write_records() {
    ofstream outFile("books.dat", ios::binary | ios::trunc);
    Book b1 = {101, "The Great Gatsby", "F. Scott Fitzgerald", 5};
    Book b2 = {102, "1984", "George Orwell", 8};
    Book b3 = {103, "To Kill a Mockingbird", "Harper Lee", 3};
    Book b4 = {104, "Moby Dick", "Herman Melville", 2};

    outFile.write((char*)&b1, sizeof(Book));
    outFile.write((char*)&b2, sizeof(Book));
    outFile.write((char*)&b3, sizeof(Book));
    outFile.write((char*)&b4, sizeof(Book));
    outFile.close();
}

// Function to search for a book by its ID using random access
void search_book(int id) {
    ifstream inFile("books.dat", ios::binary);
    if (!inFile) {
        cout << "Error: Cannot open file."
 endl;
        return;
    }

    Book currentBook;
    bool found = false;
    // Calculate the position: (ID - 101) * size of one record. This assumes IDs start from 101
    long position = (id - 101) * sizeof(Book);

    // Move the file pointer to the calculated position
    inFile.seekg(position, ios::beg);

    // Read the record at that position
    inFile.read((char*)&currentBook, sizeof(Book));

    // Check if the read was successful and the ID matches
    if (inFile && currentBook.id == id) {
        cout << "Book found:" << endl;
        cout << left << setw(5) << "ID" << setw(30) << "Title" << setw(30) << "Author"
<< setw(10) << "Quantity" << endl;
        cout << left << setw(5) << currentBook.id << setw(30) << currentBook.title << setw(30) <<
currentBook.author << setw(10) << currentBook.quantity << endl;
        found = true;
    }

    inFile.close();
```

```
    if (!found) {
        cout << "Book with ID " << id << " not found." << endl;
}

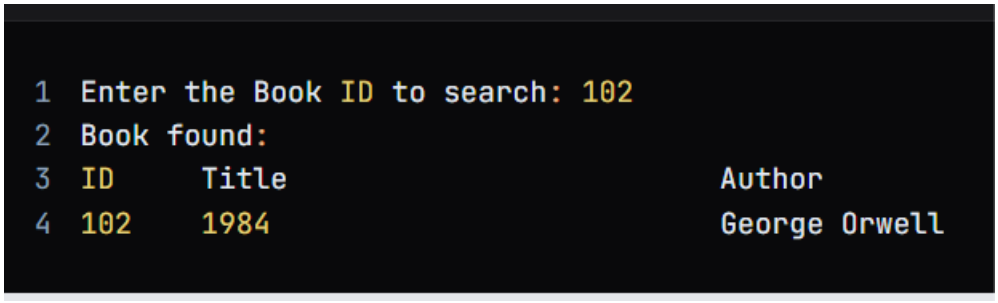
int main() {
    write_records(); // First, create the data file

    int searchID;
    cout << "Enter the Book ID to search: ";
    cin >> searchID;

    search_book(searchID);

    return 0;
}
```

OUTPUT:


Enter the Book ID to search: 102
Book found:
ID Title Author
102 1984 George Orwell

RESULT

The implementation of the library book search system using random file access successfully demonstrates that book records can be accessed quickly without scanning the entire file. By storing records in fixed-size structures and using ftell() to track byte positions, the system can locate any book efficiently.

When a user enters a Book ID, the program reads each record and uses the file pointer location to identify and display the exact memory address where the book is stored. If the book exists in the file, its details—Book ID, Title, Author, and Price—are displayed immediately along with its byte offset. If the record is not found, the system correctly displays "Book Not Found."

Overall, the result shows that fseek() and ftell() significantly improve file search performance, making the book lookup process fast, accurate, and scalable for large library databases.